Recursion

When can recursion be used?

Recursion can be used when the main problem can be solved by **solving smaller sub-problems**.

Which is very often the case.

Let's say we need a function `count_total_len`, which finds the total length of all words in a list:

```
count_total_len(["Hello", "there", "world", "!"])
```

(There are more Python-typical ways of solving this particular problem, but for practice, let's use recursion.)

What is recursion?

When using recursion, a function **calls itself from within its own code**.

```
def count_total_len(word_list):
    if len(word_list) == 0:
        return 0
    return len(word_list[0]) + count_total_len(word_list[1:])
```

```
count_total_len(["Hello", "there", "world", "!"])
```

What is recursion?

When in the process of writing a recursive function, pretend that "you already have a function that works", and that you can call.

```
def count_total_len(word_list):
    if len(word_list) == 0:
        return 0
    return len(word_list[0]) + count_total_len(word_list[1:])
```

```
count_total_len(["Hello", "there", "world", "!"])
```

What is recursion?

```
1  def count_total_len(word_list):
2      print("Count length for: ", word_list)
3      if len(word_list) == 0:
4          return 0
5      return len(word_list[0]) + count_total_len(word_list[1:])
```

```
1  count_total_len(["Hello", "there", "world", "!"])
```

```
Count length for:  ['Hello', 'there', 'world', '!']
Count length for:  ['there', 'world', '!']
Count length for:  ['world', '!']
Count length for:  ['!']
Count length for:  []
```

What is recursion?

```
1  def count_total_len(word_list):
2      if len(word_list) == 0:
3          return 0
4      result = len(word_list[0]) + \
5          count_total_len(word_list[1:])
6      print(f"{word_list} has length {result}.")
7      return result
```

```
1  count_total_len(["Hello", "there", "world", "!"])
```

```
['!'] has length 1.
['world', '!'] has length 6.
['there', 'world', '!'] has length 11.
['Hello', 'there', 'world', '!'] has length 16.
```

Base case

A base case is needed. I.e., a case when the function is **not** to call itself.

Here, the base case is when the list is empty.

```
def count_total_len(word_list):
    if len(word_list) == 0:
        return 0
    return len(word_list[0]) + count_total_len(word_list[1:])
```

```
count_total_len(["Hello", "there", "world", "!"])
```

Other version

In this case, the order doesn't matter. We can also traverse the list, starting to count length of the last word.

```
1  def count_total_len(word_list):
2      print("Checking length for: ", word_list)
3      if len(word_list) == 0:
4          return 0
5      return len(word_list[-1]) + \
6          count_total_len(word_list[:-1])
7
8  count_total_len(["Hello", "there", "world", "!"])
```

```
Checking length for:  ['Hello', 'there', 'world', '!']
Checking length for:  ['Hello', 'there', 'world']
Checking length for:  ['Hello', 'there']
Checking length for:  ['Hello']
Checking length for:  []
```

How to think

count_total_len(["Hello", "there", "world", "!"])

["Hello", "there", "world", "!"]

count_total_len(["Hello", "there", "world"]) + len("!")

15 + 1 = 16

Only care about counting the length of the last word.

We pretend that we already have a functioning "count_total_len" that can take care of the rest.

We also need a base case:

count_total_len([])

0